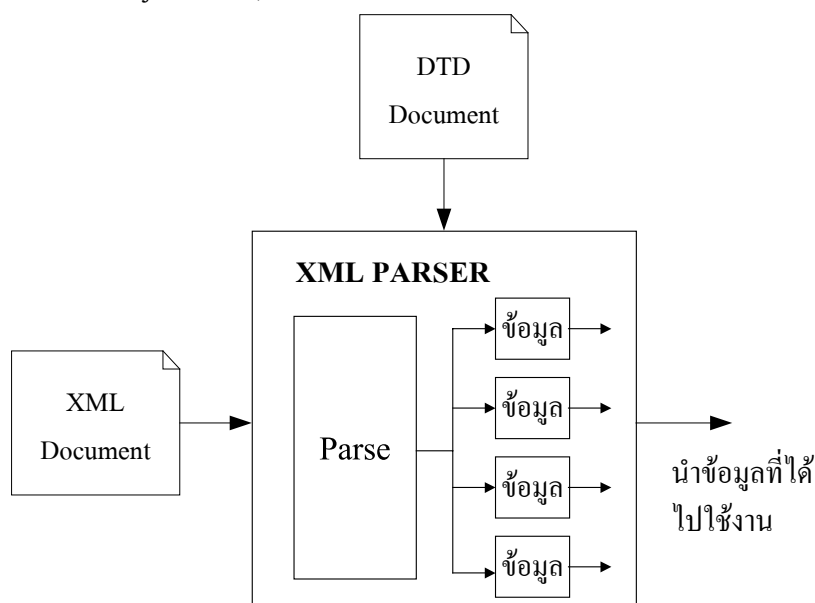


บทที่ 4

XML Parser

ก่อนที่จะนำเอกสาร XML ไปใช้งานได้นั้นจำเป็นจะต้องผ่านกระบวนการที่เรียกว่าการ “parsing” เสียก่อน โดยกระบวนการ parsing หรือการ parse เอกสารนั้นก็คือ การที่ทำให้คอมพิวเตอร์เข้าใจถึงโครงสร้างและองค์ประกอบต่างๆของเอกสารนั่นเอง และส่วนที่รับผิดชอบในการ parse เอกสารเรา จะเรียกว่า Parser ในปัจจุบันมีมาตรฐานหรือวิธีการหลักๆ อยู่ 2 อย่างได้แก่ SAX (Simple API for XML) และ DOM (Document Object Model)



รูปที่ 4-1 แสดงการทำงานของ parser

4.2 SAX (Simple API for XML)

SAX เป็นแอปพลิเคชัน โปรแกรมมิ่งอินเทอร์เฟซ (API) ที่มีลักษณะการทำงานก็คือเมื่อมีเหตุการณ์ (เช่น การร้องขอข้อมูล) ใดๆ เกิดขึ้น จะเรียกโมเดลการทำงานแบบนี้ว่า event-based model กล่าวคือ SAX จะไม่มีการสร้างภาพรวมของเอกสารไว้ก่อนเลยว่าเอกสารมีโครงสร้างเป็นเช่นไร เมื่อแอปพลิเคชันมีการร้องขอมาจึงจะรายงานหรือทำงานตามเหตุการณ์ของการทำ parsing ไปยังแอปพลิเคชันดังกล่าว

หลักการทำงานของ SAX จะมองเพียง “จุดเริ่มต้น” กับ “จุดสิ้นสุด” ของเอกสารและของอิลิเมนต์เท่านั้น และจะสนใจเฉพาะสิ่งที่ต้องการเท่านั้น

```

<?xml version="1.0"?>
<DOC>
  <ELEMENT1>value 1</ ELEMENT 1>
  < ELEMENT 2>value 2</ ELEMENT 2>
</DOC>
  
```

SAX จะมองเอกสารข้างต้นไปที่ละบรรทัดๆ ในลักษณะของ linear events ดังนี้

start document

start element: DOC

start element: ELEMENT 1

characters: value 1

end element: ELEMENT 1

start element: ELEMENT 2

characters: value 2

end element: ELEMENT 2

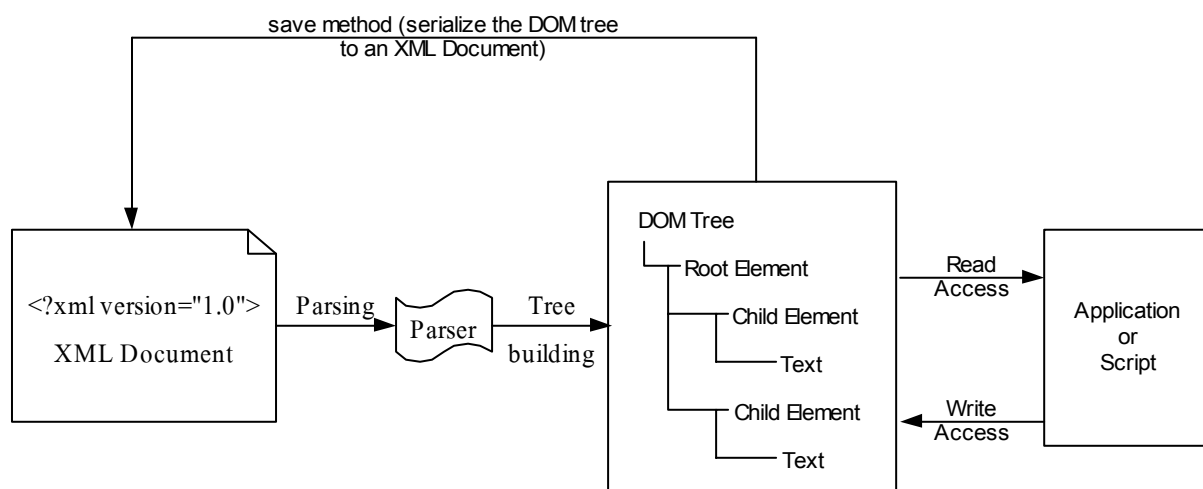
end element: DOC

end document

จากตัวอย่าง SAX จะทำการพาร์สเอกสารตั้งแต่บรรทัดแรก จนถึงบรรทัดสุดท้ายแบบเส้นตรง

4.3 Document Object Model

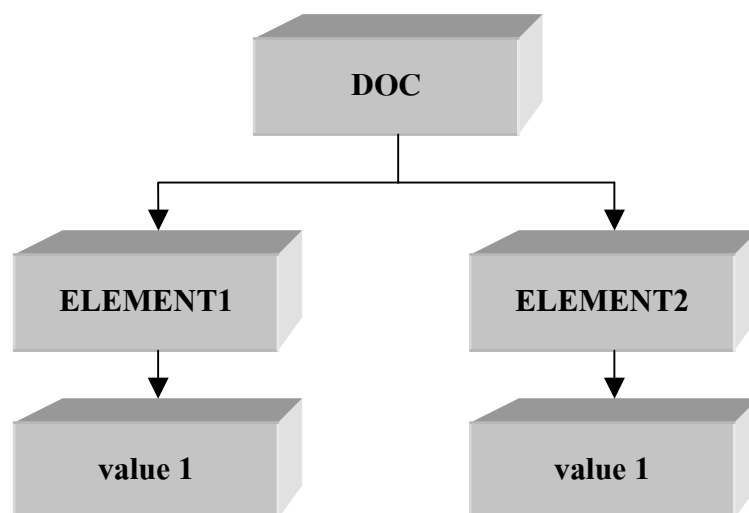
DOM คือแอปพลิเคชัน โปรแกรมมิ่งอินเทอร์เฟซ (Application Programming Interface, API) สำหรับเอกสาร XML ซึ่งจะนำเสนอข้อมูลของ XML ในลักษณะของทรี (tree) เพราะว่าการ traversal และการจัดการ ของโครงสร้างแบบทรี ง่ายต่อการเขียนโปรแกรม DOM จะอ่านเอกสาร XML ลงไปใน หน่วยความจำ (memory) และเก็บข้อมูลทั้งหมดอยู่ใน โหนด (node) ดังนั้นการเข้าถึงข้อมูลจะเร็วมาก ระดับบนสุดของทรีเราจะเรียกว่า “document Element” โอลิเมนต์ดังกล่าวจะแตกสาขาออกไปเป็นโอลิเมนต์ย่อยๆ ตั้งแต่ 1 โอลิเมนต์ขึ้นไป โดยโอลิเมนต์ย่อยที่แตกออกไปเราจะเรียกว่า “child node” ซึ่งรูปแบบการทำงานของ parser ที่เป็น DOM สามารถแสดงได้ดังรูปที่ 4-2 ซึ่งในโครงการนี้ได้เลือกใช้ DOM เป็นตัวพาร์สเซอร์ เพราะสามารถจัดการ (เช่น การแก้ไข หรือการลบข้อมูล) กับข้อมูล XML ที่เก็บอยู่ใน โหนดได้ง่ายกว่า การทำด้วย SAX พาร์สเซอร์



รูปที่ 4-2 แสดงการทำงานของ parser ที่เป็น DOM

จากตัวอย่างด้านล่างนี้ เมื่อผ่านกระบวนการ parsing โดย DOM จะทำการมองเอกสารออกมาเป็นทรีดังนี้

```
?xml version="1.0"?>
<DOC>
  <ELEMENT1>value 1</ ELEMENT 1>
  < ELEMENT 2>value 2</ ELEMENT 2>
</DOC>
```

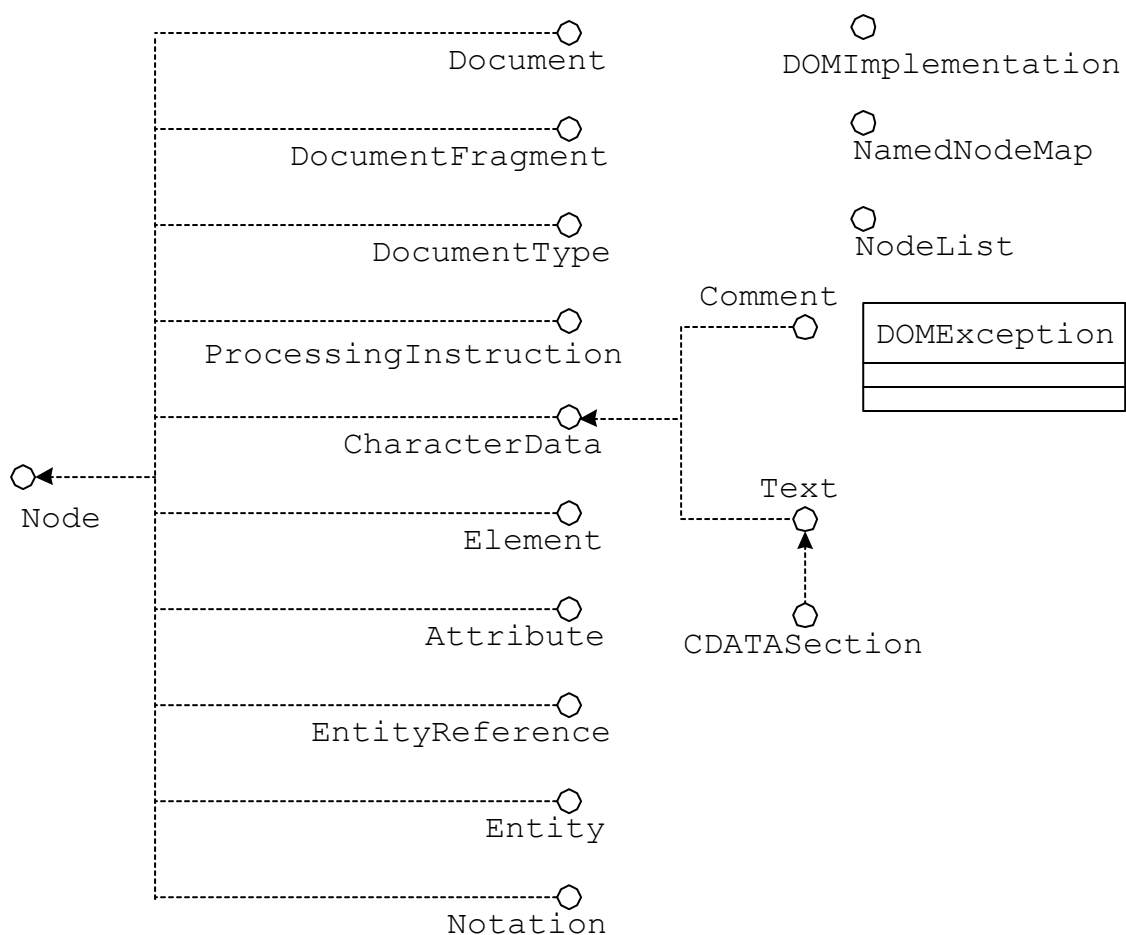


รูปที่ 4-3 แสดงลักษณะการมองข้อมูลเป็นทรีใน DOM

การจะเข้าถึงข้อมูลในแต่ละโหนดของทรี ถ้าเราทำงานภายใต้การทำงานของบราวเซอร์ อย่างเช่น Internet Explorer ตั้งแต่ version 5 เป็นต้นไปจะมีการฝัง XML Parser ไว้อยู่แล้ว แต่ถ้าเราต้องการจะเข้าถึงข้อมูลโดยใช้ภาษา Java เราจะต้องมีคลาสของ DOM Parser ก่อน ในที่นี้เราจะใช้ Xerces ของ Apache โดยในคลาสของ Xerces นี้จะได้อัปเดตเตรียมฟังก์ชันต่างๆ ที่จำเป็นต้องใช้ในการพาร์สและจัดการกับข้อมูล

4.3.1 การใช้งาน DOM Tree

เมื่อได้ DOM Tree จากกระบวนการ parsing แล้ว เราสามารถที่จะย้าย หรือปรับเปลี่ยนโครงสร้างของทรี หรือเข้าถึงข้อมูล และนำข้อมูลจากทรีของข้อมูล XML มาแสดงผล การที่เราจะทำอะไรกับข้อมูลในทรีนั้น เราจะต้องเข้าใจ object พื้นฐานของข้อมูลใน XML ที่จะสามารถเข้าถึงได้ โดยอินเทอร์เฟซเหล่านี้ได้แสดงในรูปที่ 4-3 ซึ่งเราจะจัดการกับข้อมูลทั้งหมดภายใต้ DOM tree นี้



รูปที่ 4-3 UML class model of DOM Level 2 core interfaces and classed

จากรูปเราจะให้ความสนใจกับ Node interface เป็นพิเศษ จะสังเกตว่ามันเป็น base interface สำหรับ interfaces อื่นๆ จึงสามารถที่จะเขียน method กระทำกับโหนดครอบคลุมทุกประเภทของโหนดในโครงสร้างแบบ DOM